Test-Time Compute as Resource Allocation

Arihan Iyer, Kushagra Mishra, Jatin Shihora, Rohit Puthucode

Speculation: Guess Now, Verify Later

1 A slow step blocks the next one → **2 Guess its result cheaply** → **3 Start the next step in parallel** → **4 Verify: keep if right, discard if wrong**

Where it is used	What is guessed	Who guesses	Who verifies
Speculative decoding	The next K tokens	Small draft model	Big model, in one parallel pass
SPEX (paper 1, today)	Next tree branches to expand	Search rule on current rewards	The real reward
Speculative Actions (paper 2)	The agent's next action	A fast model	The slow agent or real API

Speculation pays off when guesses are often right and extra work is nearly free

C. Chen et al., "Accelerating Large Language Model Decoding with Speculative Sampling," arXiv:2302.01318, 2023.

Breaking the Reward Barrier: Accelerating Tree-of-Thought Reasoning via Speculative Exploration

Shuzhang Zhong, Haochen Huang, Shengxuan Qiu,
Pengfei Zuo, Runsheng Wang, Meng Li
OSDI ’26

What Is Tree-of-Thought Reasoning?

Chain-of-Thought

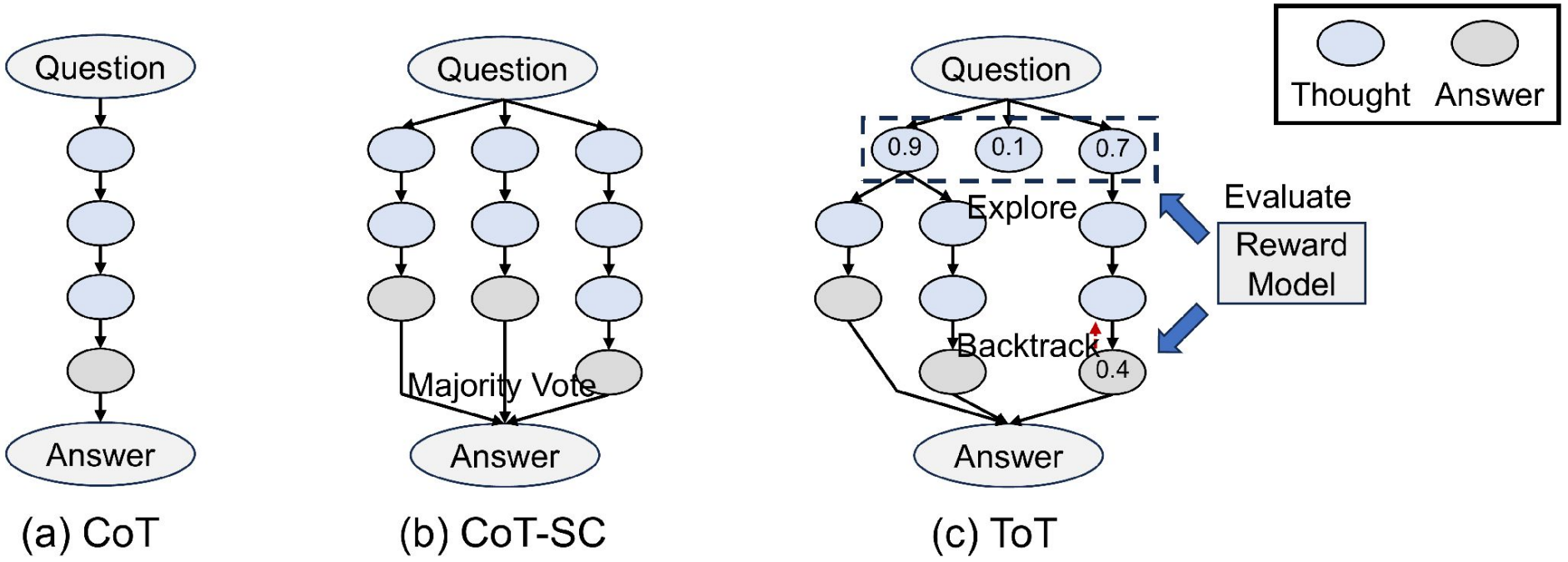
One chain of reasoning steps: reasoning goes deeper

Self-consistency

Several chains, then a majority vote: reasoning goes wider

Tree-of-Thought

A reward model scores each step, so good branches grow and bad ones are abandoned



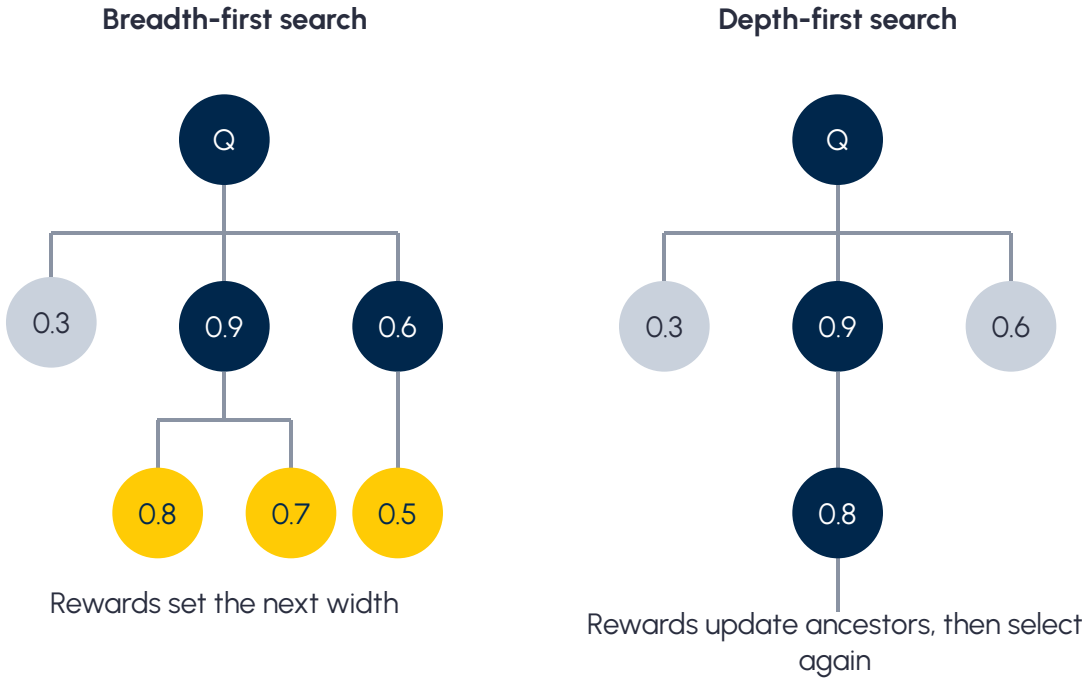
S. Yao et al., "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," NeurIPS 2023.

Tree-of-Thought Reasoning Searches over Steps

Each node is one reasoning step
A path from question to answer is one solution; a reward scores each step.

BFS expands a whole depth at a time
Rewards decide how many children each node expands, as in REBASE.

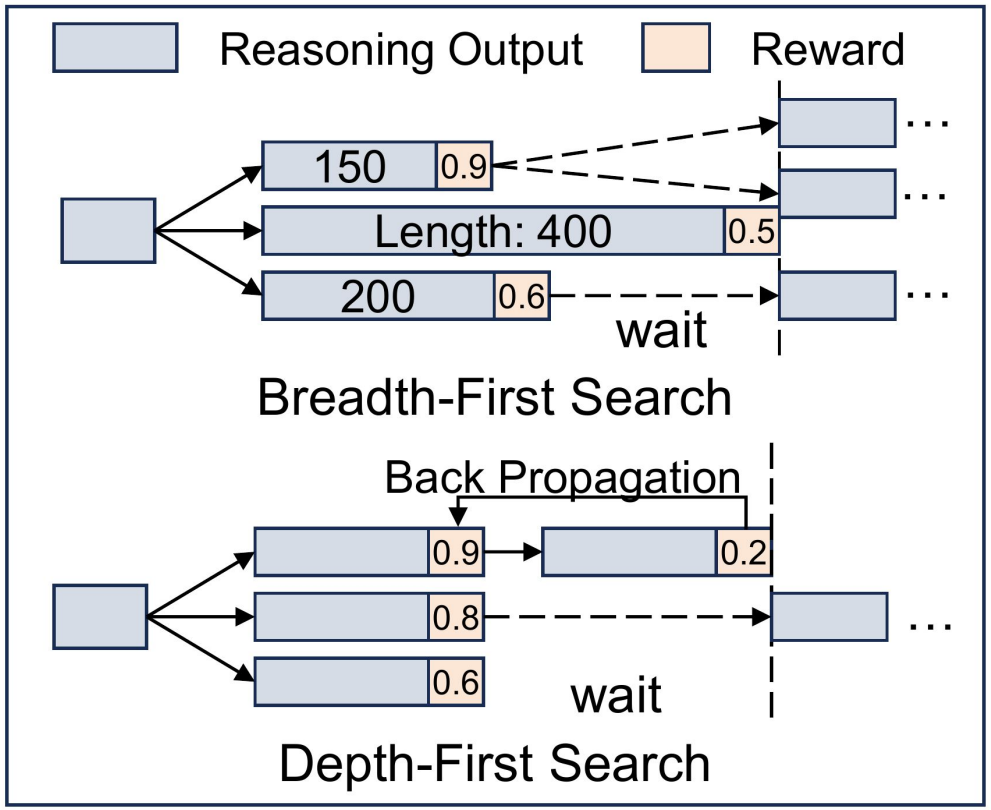
DFS revisits nodes through rollouts
MCTS back-propagates rewards, then selects again (RSTAR-MCTS, REST-MCTS).



Illustrative reward scores

Serving systems such as vLLM and SGLang were built for one path, not trees

The Reward Dependency Barrier Stalls Tree Search



Breadth-first: wait for the slowest branch

The 150- and 200-token branches finish early, then idle until the 400-token branch is scored: the next level needs every reward.

Depth-first: wait for the previous rollout

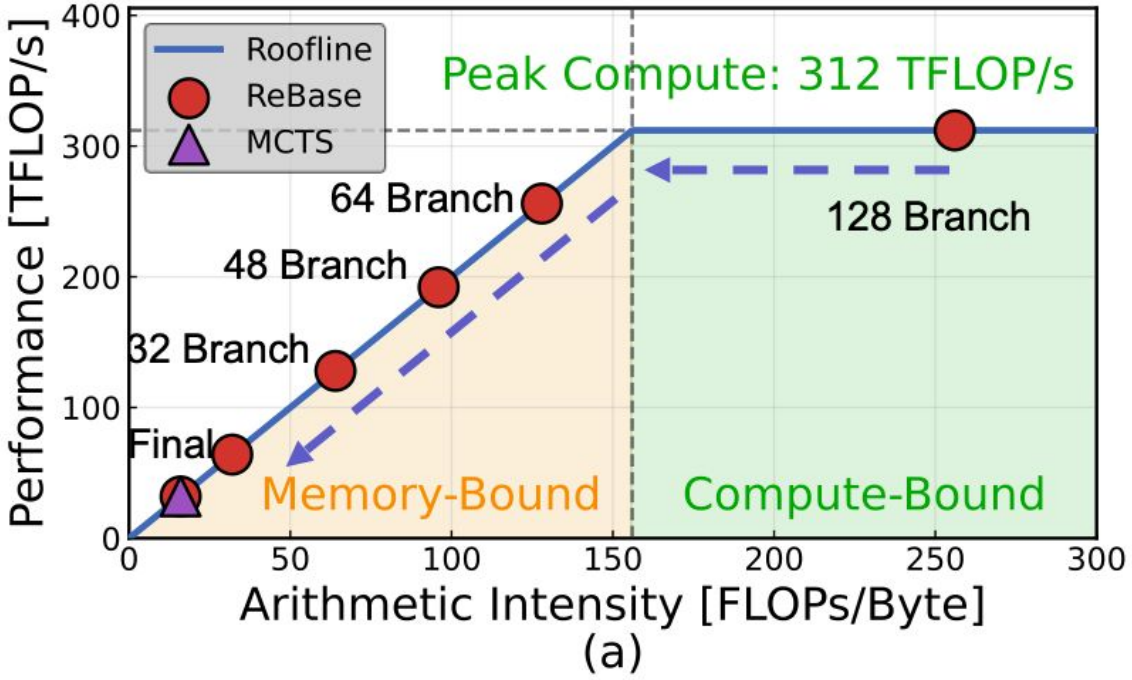
The next node is picked only after the 0.2 reward back-propagates, so one branch decodes at a time: batch size ≈ 1 from the start.

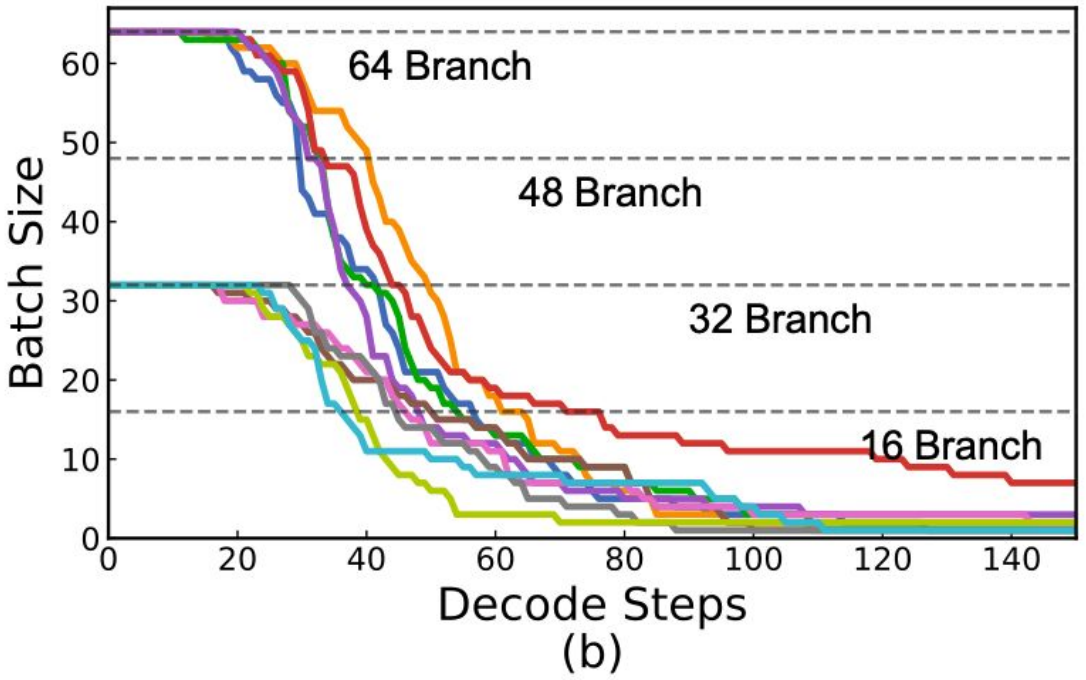
Long tails make it worse

Node lengths vary by up to ~1,000 tokens, so stragglers are common.

Both paradigms leave the GPU idle while waiting for another branch's reward signal

The Reward Barrier Keeps Decoding Memory-Bound





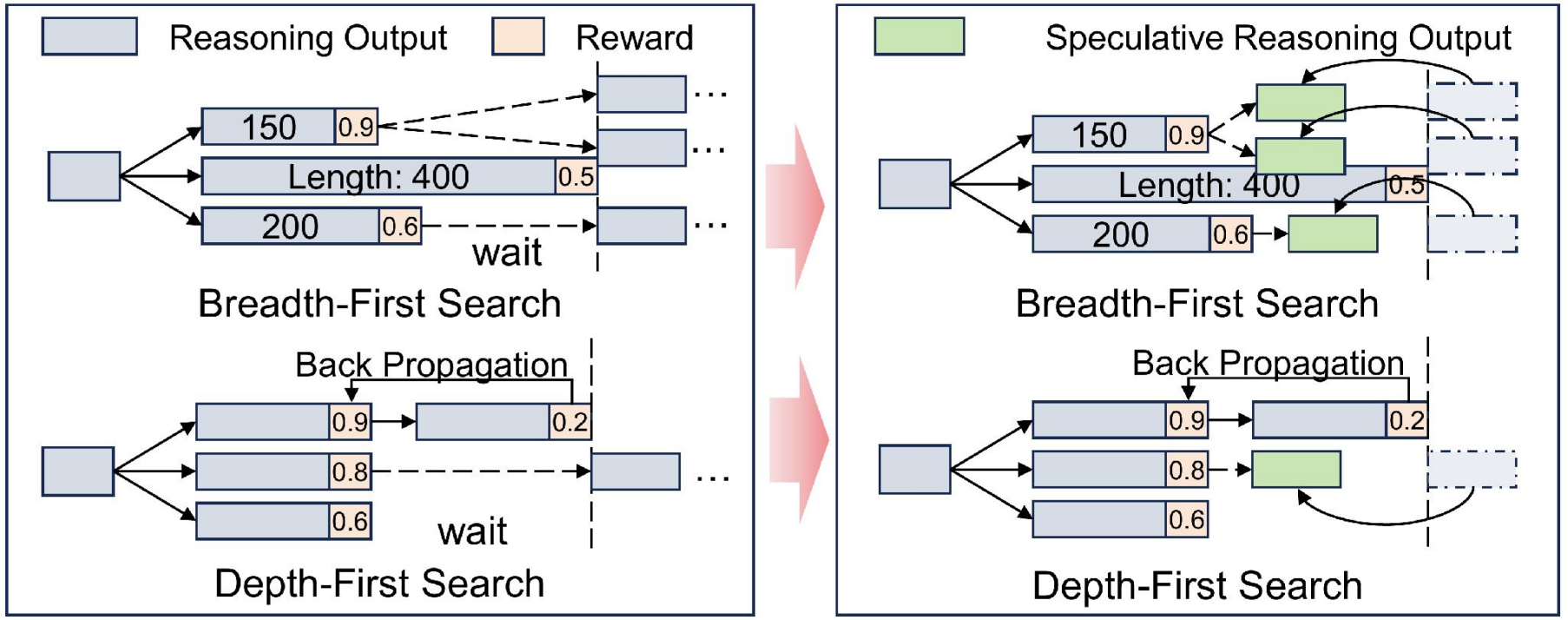
(a) DFS never leaves the memory-bound region

MCTS decodes ~1 branch at a time and reaches ~10% of the 312 TFLOP/s peak: each step reloads all weights for little math.

(b) BFS slides into it

64 branches start near the ridge, but they finish at different times, so the batch shrinks within ~60 decode steps.

Speculate on Branches Before the Reward Arrives



SPEX expands likely-needed branches (green) before their reward-gated turn

Right guesses finish early; wrong ones are cancelled (BFS) or kept for a later rollout (DFS).

Speculating in ToT Raises Three Challenges

1

How to predict future-needed branches?

MCTS can revisit any node in the tree, so the branch it needs next is hard to name before the reward arrives.

2

How to allocate the speculative budget?

Spare capacity must be split across the branches of many query trees, which differ in candidate count and hit odds.

3

How to handle skewed search trees?

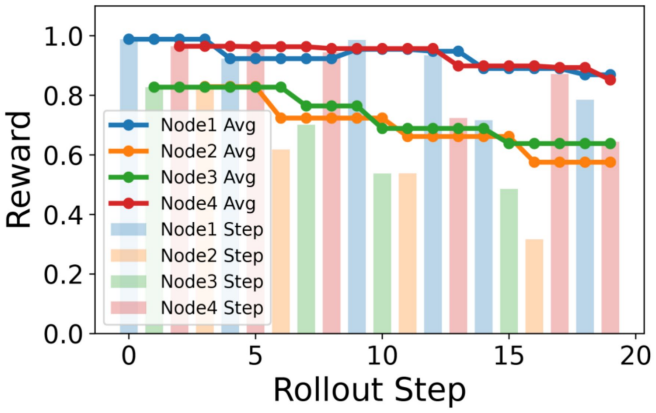
When one deep branch dominates, there is little to run in parallel, and deep nodes are the hardest to predict.

SPEX answers each challenge with an observation about how ToT behaves

Three Observations Guide the Design

Challenge 1: which branches?

Node rewards change slowly across rollouts



Intra-query selection

Challenge 2: who gets the budget?

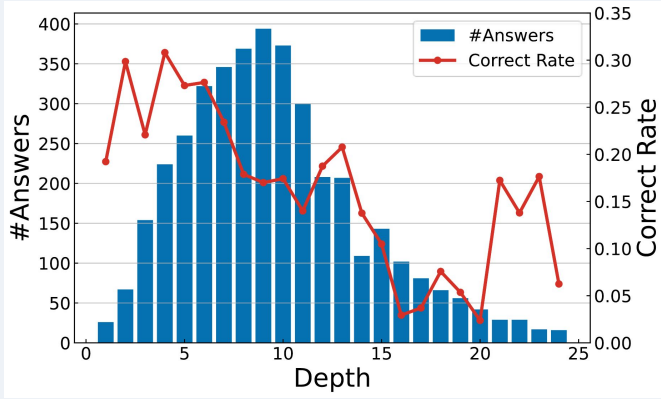
Shared computation makes speculation cheap

Decoding is memory-bound, so a speculative branch rides on weight loads the GPU already makes, and sibling branches share KV-cache prefixes. How much a query can reuse decides where speculation pays.

Inter-query budget allocation

Challenge 3: skewed trees?

Most answers, and most correct ones, are shallow



Adaptive early termination

SPEX Answers Three Challenges with Three Techniques

1 **Intra-query speculative selection**

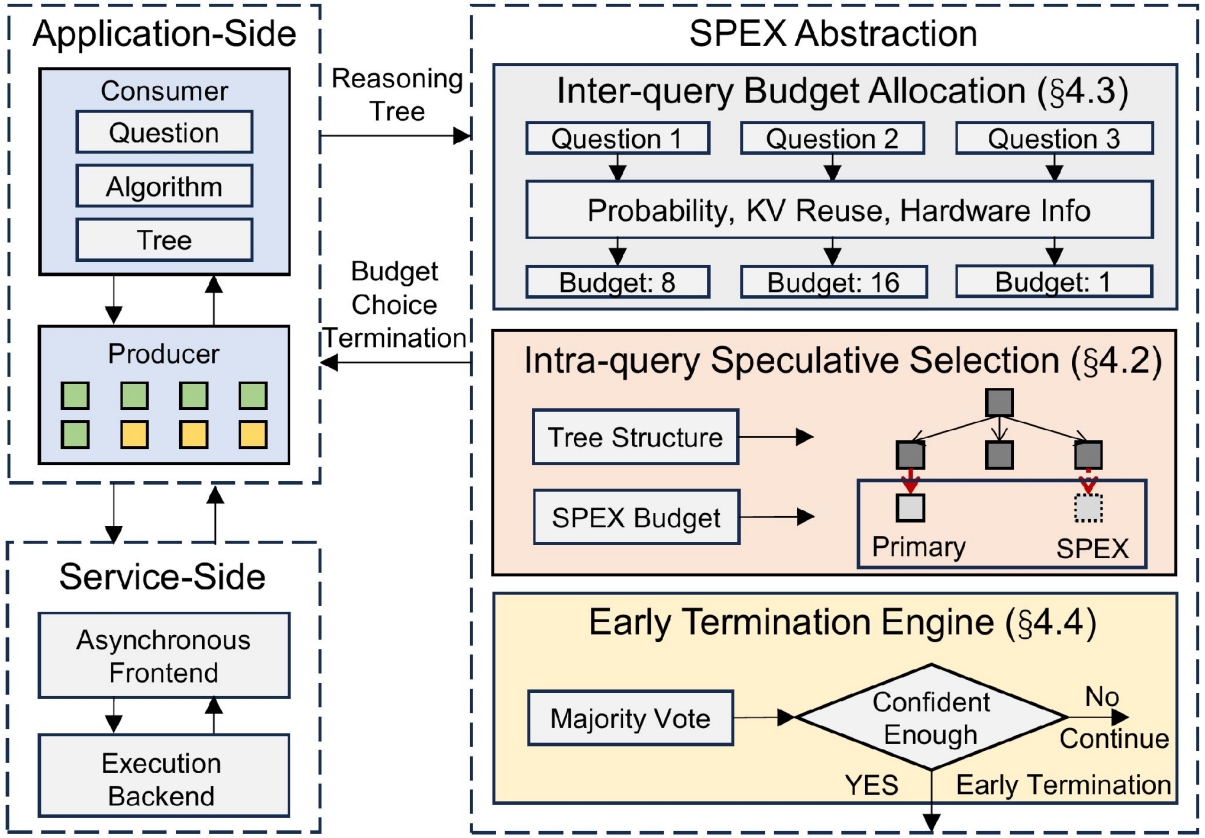
Predicts which branches the search will need and expands them before rewards arrive.

2 **Inter-query budget allocation**

Splits the spare GPU capacity across query trees by expected benefit.

3 **Adaptive early termination**

Stops a deep, skewed search once the leading answer is clear.



Intra-Query Selection for DFS: Simulate Future Picks

Simulate the next k picks

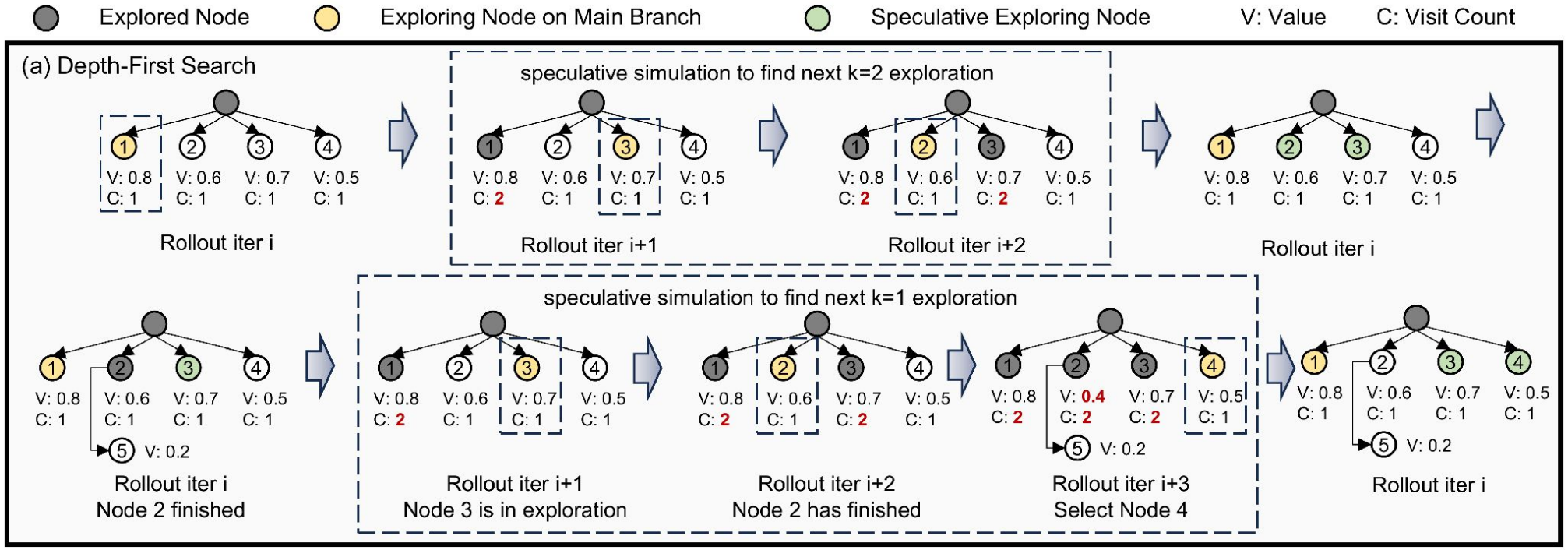
Apply the UCB rule to a temporary copy of the tree.

Bump counts, not values

Each simulated pick raises its visit count C, so its bonus shrinks and the next pick moves on.

Skip and reuse

Skip nodes already speculated; fold in finished results (node 2: V 0.6 → 0.4).



Intra-Query Selection for BFS: Fill Idle Slots

Budget = finished nodes

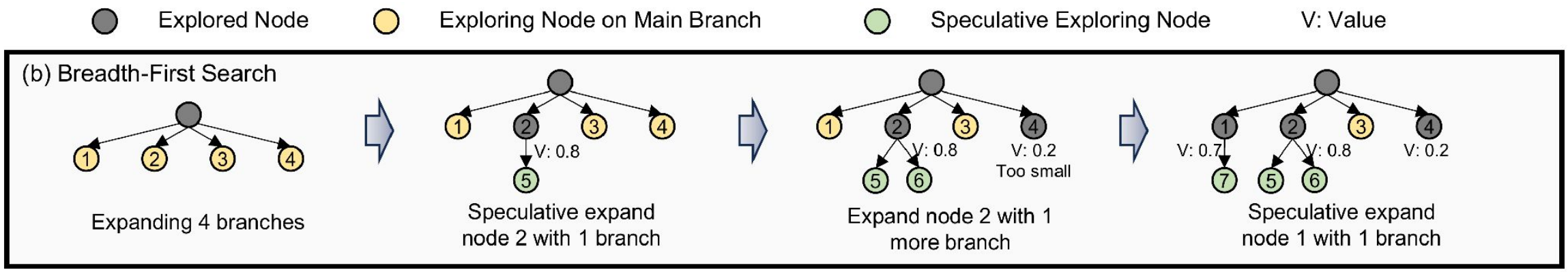
Each short branch that finishes frees a slot while the stragglers keep decoding.

Follow the baseline policy

Split slots with REBASE's reward softmax: node 2 (V 0.8) gets more children than node 4 (V 0.2).

Handle wrong guesses

BFS cancels branches that fall off the real frontier; DFS keeps them for later rollouts.



Inter-Query Budget Follows Expected Utility

Useful branches

Branches query q could speculate on (C_q) × how often its guesses get used (P_q)

Roofline payoff

Memory-bound decoding: a branch is worth the weight (S_w) and KV-cache (S_KV) reads it shares

$$\text{score}_q = C_q \times P_q \times (S_w + S_{KV(q)})$$

$$k_q = \min(C_q, k_{total} \times \text{softmax}(\tau \cdot \text{score}_q))$$

Global budget

Slots until the batch reaches the roofline ridge and turns compute-bound

Share by score

Softmax splits k_total by score (τ sets how sharply); min() caps it at C_q

score_q: expected benefit of speculating for query q · k_q: speculative slots that query q receives

Early Termination Prunes Deep Branches

Lead of the top answer

Total reward behind the leading candidate minus the runner-up

Margin scales with the rival

α (0.5) times one average runner-up vote: stronger rivals need a bigger lead

$$\text{conf}_{1\text{st}} - \text{conf}_{2\text{nd}} > \alpha \cdot \text{avg}(w_{2\text{nd}})$$

$$\text{checked once} \quad n \geq t$$

Why it is safe

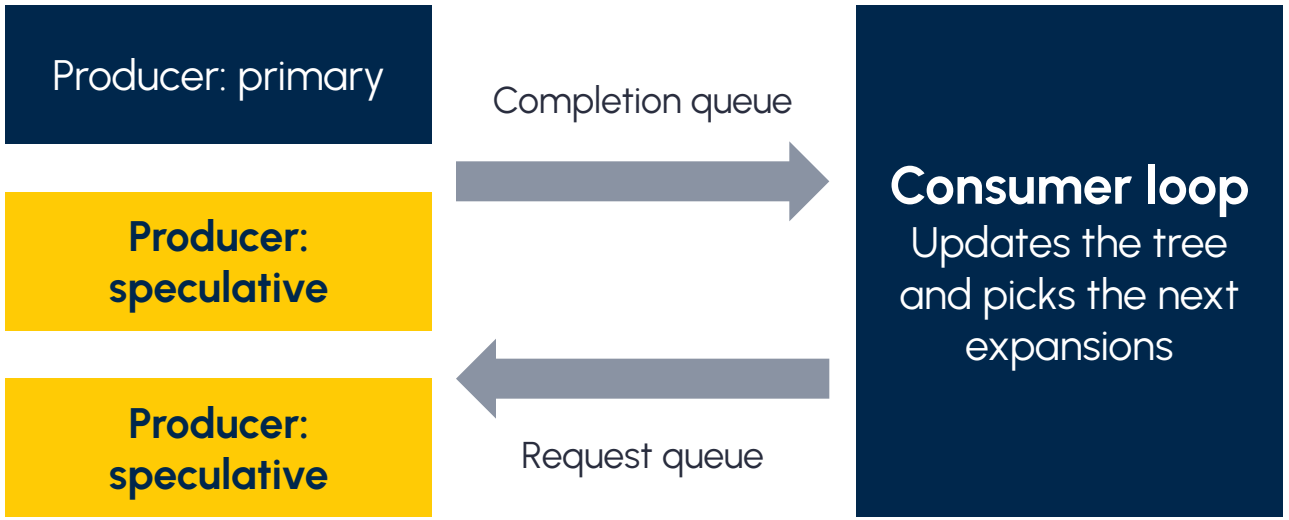
Most answers, and most correct ones, come from shallow depths (Observation 3)

Enough evidence first

n answers so far must reach a minimum t before stopping

conf_1st, conf_2nd: summed reward weights w_i of the answers voting for the top two candidates

Producers and a Consumer Overlap Speculation



Non-blocking
Primary progress never waits on speculative work.

Opportunistic
Idle producers take speculative branches that share a prefix.

Algorithm-agnostic
One loop serves both BFS and DFS search algorithms.

Built on SGLang with asynchronous responses and Python coroutines

Evaluation Setup

Models and hardware
Llemma-7B and DeepSeek-R1-8B on A6000 GPUs; Llemma-34B and Qwen3-30B-A3B on A100 GPUs.

Workloads
GSM8K and MATH-500 for Llemma; AIME 2024 and 2025, BRUMO and HMMT-Feb25 for reasoning models.

Algorithms and metrics
REST-MCTS, RSTAR-MCTS, REBASE and ETS, each in two configurations. Throughput and speedup.

SPEX Speeds Up DFS 1.8-3× and BFS 1.2-1.9×

1.8-3×

DFS algorithms: REST-MCTS and RSTAR-MCTS, average speedup

1.2-1.9×

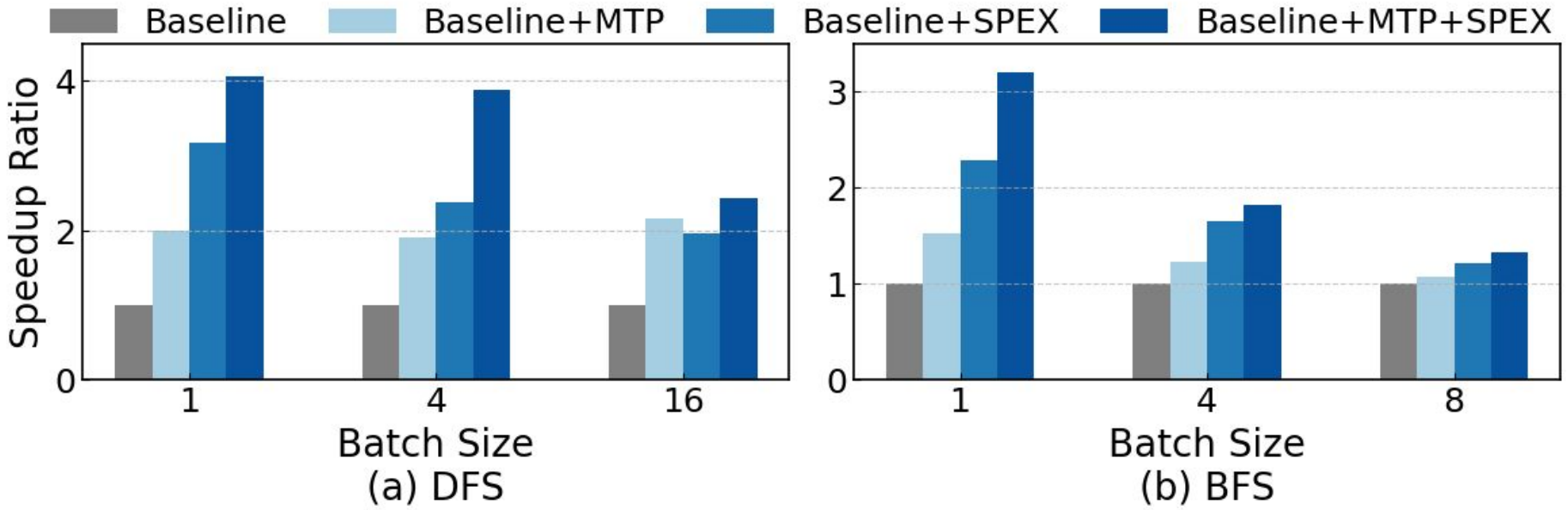
BFS algorithms: REBASE and ETS, average speedup

Speedup is largest at small batch sizes
Gains shrink as more queries share the GPU and decoding turns compute-bound.

DFS speedup grows with more answers
More rollouts give more nodes to speculate on; wider BFS shrinks the gain.

DFS gains most because its branch-level parallelism is lowest

SPEX Composes with Token-Level Speculative Decoding



(a) RSTAR-10 (b) REBASE-8 · DeepSeek-R1-8B with its built-in MTP module

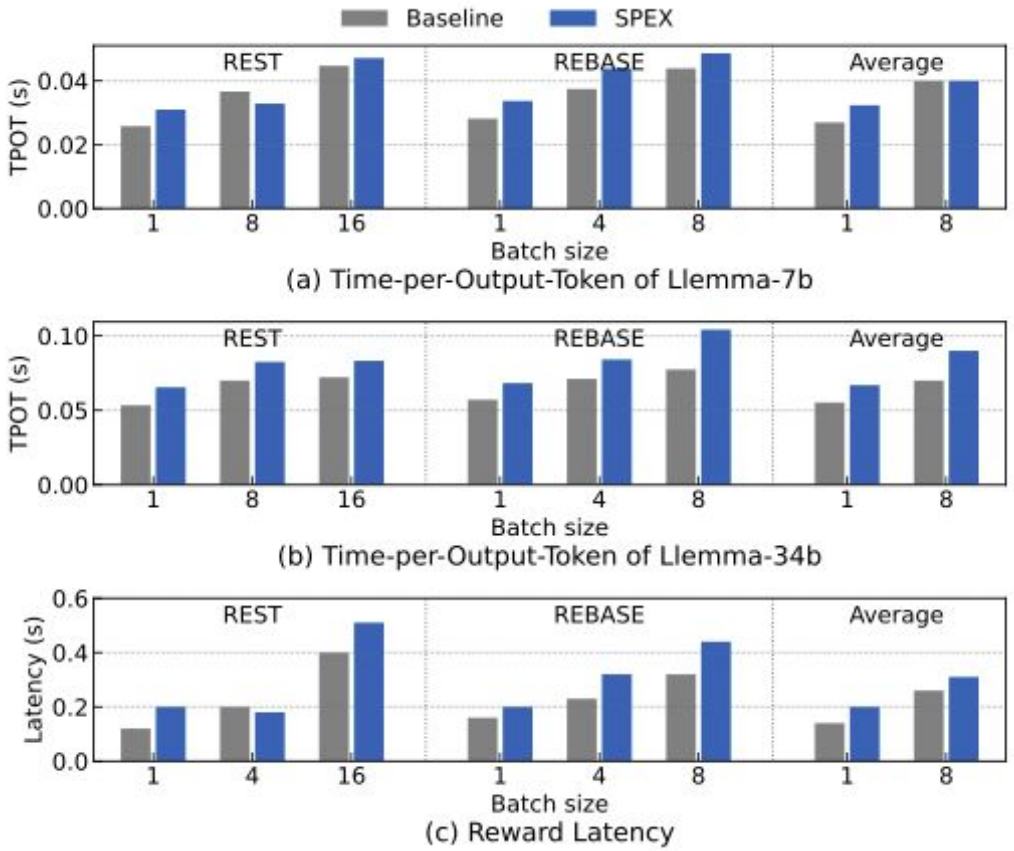
Different levels of speculation

MTP guesses the next tokens of one path; SPEX guesses which branches to expand.

Batch size decides the gain

Combined ~4.1× (DFS) at batch 1; both shrink as the GPU turns compute-bound.

Speculation Makes Each Token Slower



Time per output token rises 15–35%

Speculation enlarges the batch. The paper says within 15%, but the bars reach higher in some cases (Llemma-34B, REBASE, batch 8).

Reward scoring barely slows

Reward-model latency grows by about 0.1 s, small next to seconds of generation per step.

Worst where gains are smallest

Large models at large batch sizes are already near compute-bound, so the overhead eats into the speedup.

Strengths and Limitations

Strengths

- Breaks the barrier in both DFS and BFS
- Splits budget by KV and weight reuse
- Accuracy unchanged or slightly higher (Table 2)
- Composes with token-level speculation

Limitations

- The budget ignores KV memory capacity
- Gains shrink at large batch sizes
- About 15-35% slower per token; reward delay ~0.1 s
- Skewed deep trees stay hard to speculate
- Early termination alone is not tested

A useful ToT serving policy, strongest at small batch sizes

Code: https://github.com/shuzhangzhong/SPEX

Speculative Actions: A Lossless Framework for Faster Agentic Systems

Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, Tianyi Peng

ICLR 2026

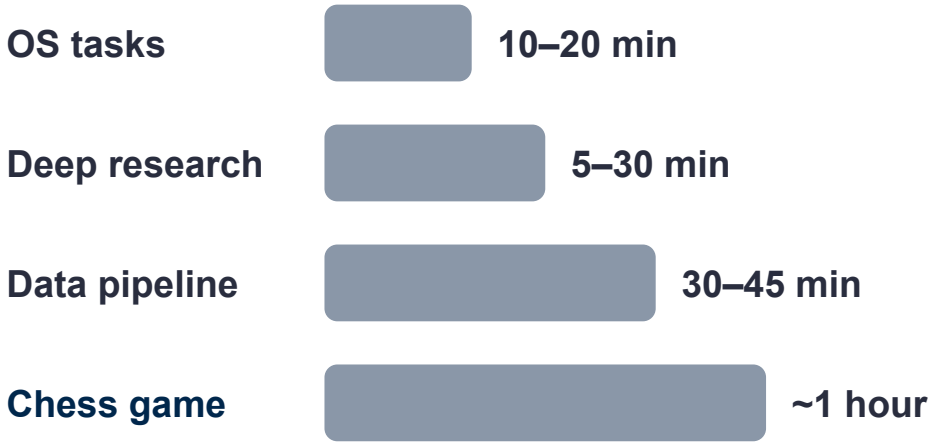
The Problem: Strictly Sequential Agents



LLM Call Tool / Human

Strictly Sequential: Each call starts only after the previous one returns

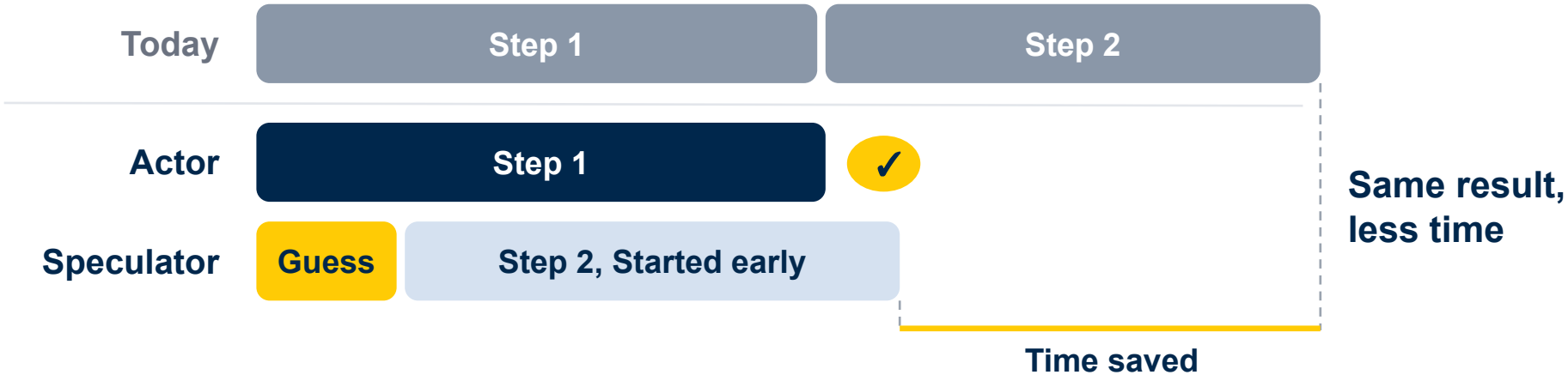
One agent run takes minutes to hours



THE QUESTION

Must an agent interact with its environment in a strictly sequential manner?

The Solution: Speculative Actions



A **Actor**
Slow, Authoritative

GPT-5 high | APIs | Environment | Human

Verify: Match → Keep, Miss → Discard

Its output is the ground truth

Guess ⟷ **Verify**

S **Speculator**
Fast, Cheap

Small LLM | Low-effort prompt | Heuristics

Guess the Next Action and Start it early

Predicts the next action, inputs and result

Design Goal: Faster, but Never Wrong

Lossless: The final result is identical to the normal, step-by-step agent

1 Semantic guards

The Actor confirms a guess before anything is committed

2 Safety envelopes

Only speculate on actions that are reversible, repeatable or sandboxed

3 Repair paths

Wrong guess? Roll it back, or compensate (e.g., refund or replace)

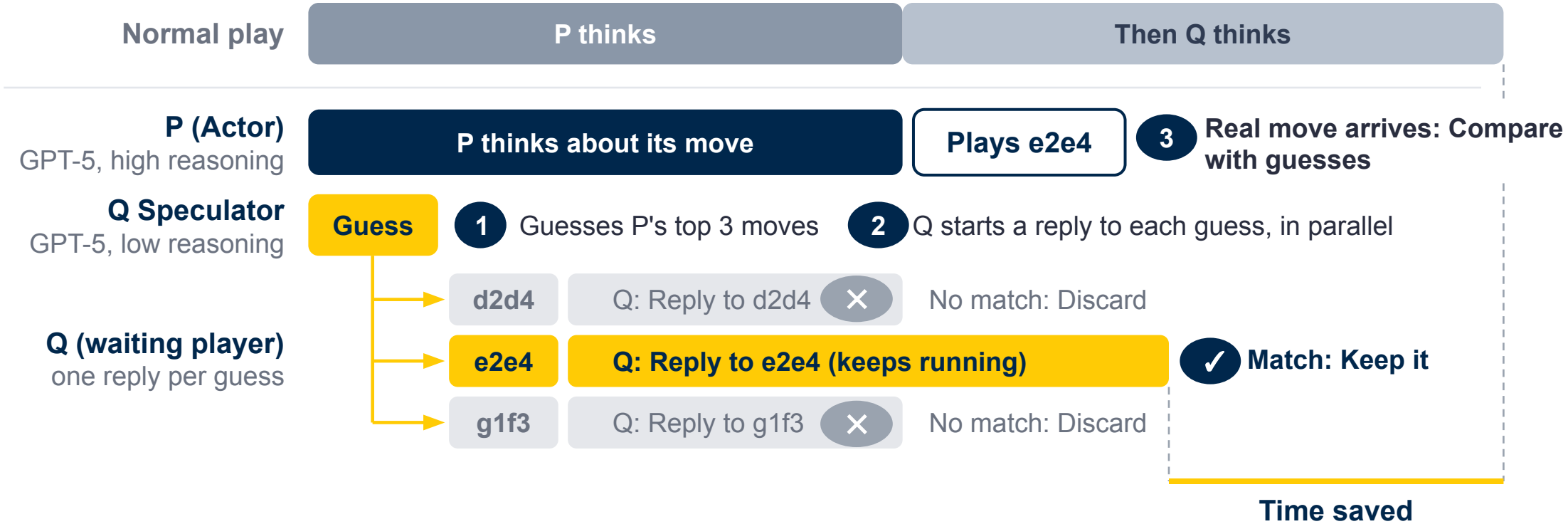
✓ Safe to speculate

Web search, read-only lookups, chess analysis, OS settings that can be overwritten

✗ Wait for the Actor

Irreversible actions: placing orders, deleting records

Chess Example: When the Guess Is Right



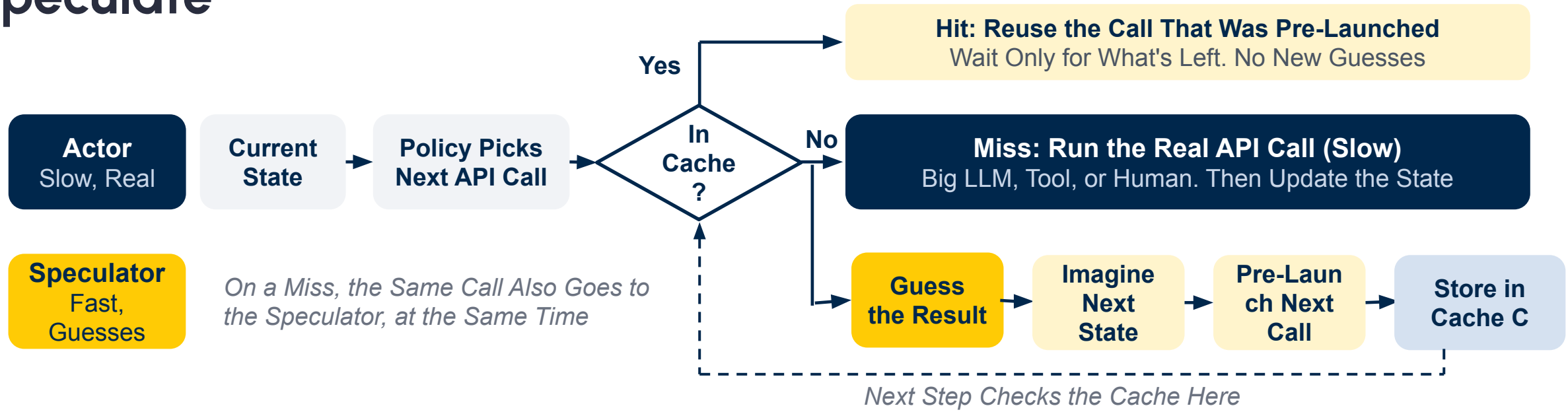
Result: Q replies earlier than in normal play. Same moves, less time.

Chess Example: When the Guess Is Wrong



Result: Same finish time as normal play. Nothing saved, nothing lost.

The Framework (Algorithm 1): Check the Cache First, Then Speculate



Why It's Faster



What the Theory Shows

With k Guesses, a Hit Happens with p(k) = 1 − (1 − p)^k

One-Step Speculation Saves up to 50% of Time; Multi-Step Can Go Further

Four Test Environments

1 Chess

Lossless

Benchmark
TextArena

Bottleneck
Long reasoning on every move

Speculator guesses
The opponent's next move

Models
GPT-5 high (Actor), GPT-5 low (Speculator)

2 E-commerce

Lossless

Benchmark
τ-bench (retail)

Bottleneck
Tool calls while the shopper waits

Speculator guesses
Shopper's next request + its API calls

Models
GPT-5 nano / mini, Gemini 2.5 Flash

3 Multi-hop QA

Lossless

Benchmark
HotpotQA (ReAct agent)

Bottleneck
Slow Wikipedia lookups

Speculator guesses
Search result, then the next search

Models
Gemini 2.5 Flash Actor, small fast Speculators

4 OS tuning

Lossy extension

Benchmark
Linux scheduler (sysbench)

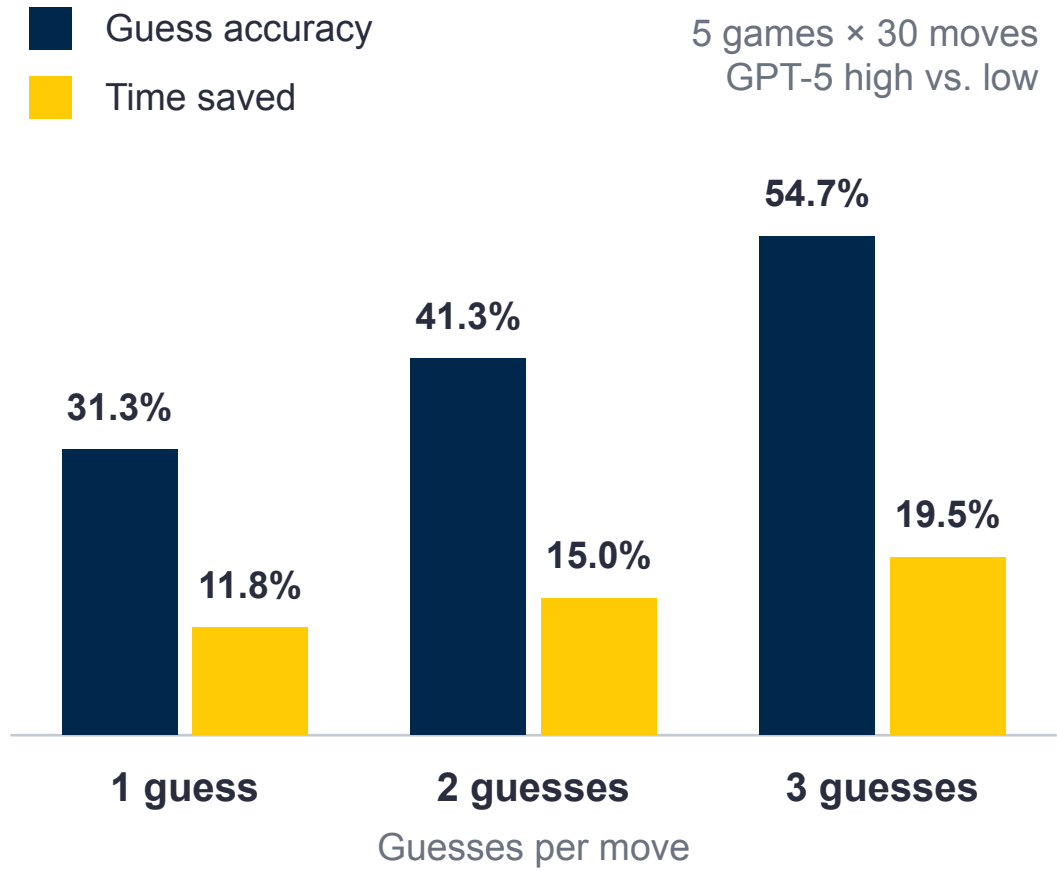
Bottleneck
Actor needs 10–15 s to react

Speculator guesses
Next scheduler setting, every 1 s

Models
Fast + slow LLM tuners

Chess: Q Prepares While P Thinks, ~20% Saved

1. **Predict**
   Speculator guesses P's top 3 moves, fast
2. **Prepare**
   Q starts a reply to every guess, in parallel
3. **Check**
   P plays: does its move match a guess?
4. **Commit or restart**
   Match: Q's reply is ready. Miss: Q thinks as usual.



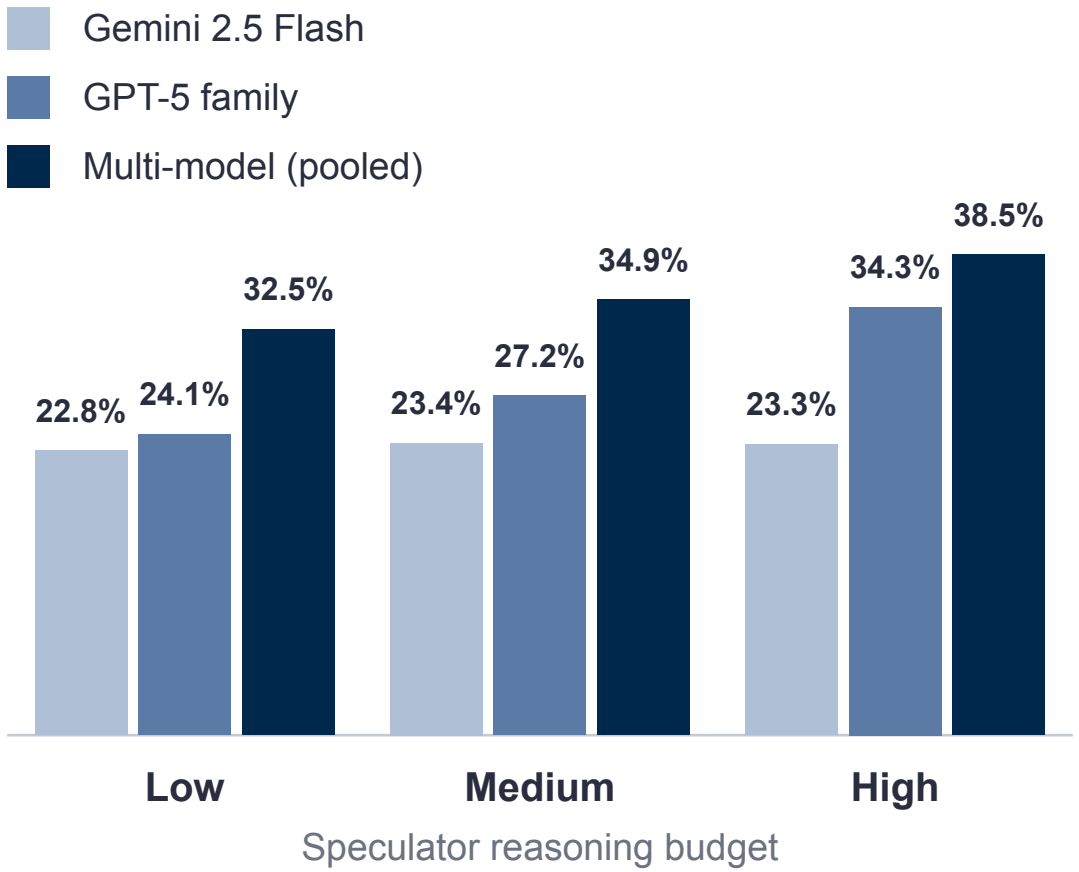
E-commerce: Pooled Speculators Predict Up to 38.5%

1. **Predict the shopper's next message**
2. **Predict the API calls it needs (get_order_details…)**
3. **Run them while the shopper types (~30 s vs. 2–3 s)**
4. **~1 in 3 turns: the answer is ready on send**

Single model
GPT-5 mini
27.2%
of calls, medium budget

Pooled models
GPT-5 mini + Gemini Flash
34.9%
of calls, medium budget

Different models guess differently, so pooling covers more

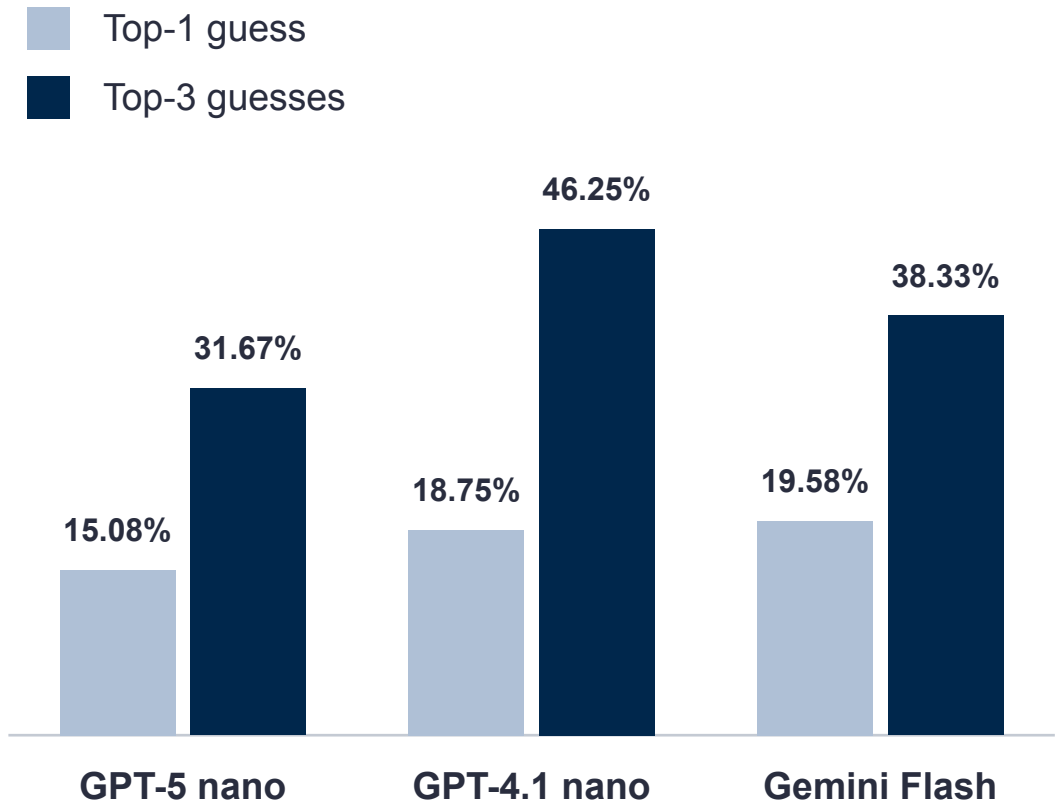


Multi-hop QA: Up to 46% of Next Searches Predicted

Actor	**Search(A)**	*waiting…*	✓
Speculator	**guess page**	**next: Search(B)**	

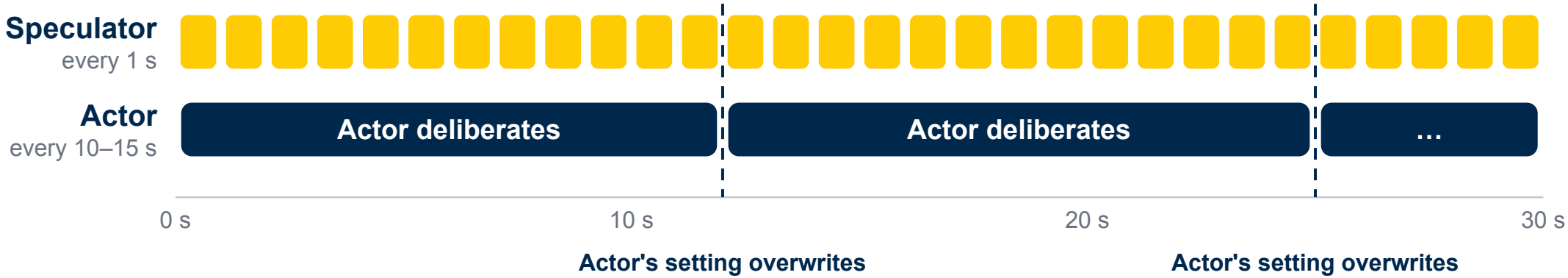
While Wikipedia answers, the Speculator guesses the page, and the Actor already picks the next search from that guess.

1. **Strict check**
   next call must match exactly: same API, same words
2. **Top-3 guesses help a lot**
   accuracy roughly doubles vs. top-1
3. **Simpler models match more often**
   their queries are phrased more predictably



Speculator model (Actor: Gemini 2.5 Flash)

Lossy Mode: Act Now, Let the Actor Overwrite



Actor-only

Deliberate but slow: one new setting every 10–15 s

Speculator-only

Fast (one setting per second) but shallow reasoning

Speculator + Actor

Speculator acts between Actor decisions; the Actor reads a compressed log of the Speculator's (setting, result) pairs

✓ **Safe without rollback: a scheduler setting can always be replaced by the next one**

OS Tuning: The System Can't Wait for the Actor

Background: the Linux CPU scheduler

CFS gives every task a fair share of the CPU. Its knob **min_granularity** is the shortest time slice a task runs before it can be preempted.



← Smaller
more responsive, more switching overhead

Larger →
more throughput, worse latency

Workload: sysbench cpu, 16 threads on 2 cores
Goal: minimize p95 latency

The problem

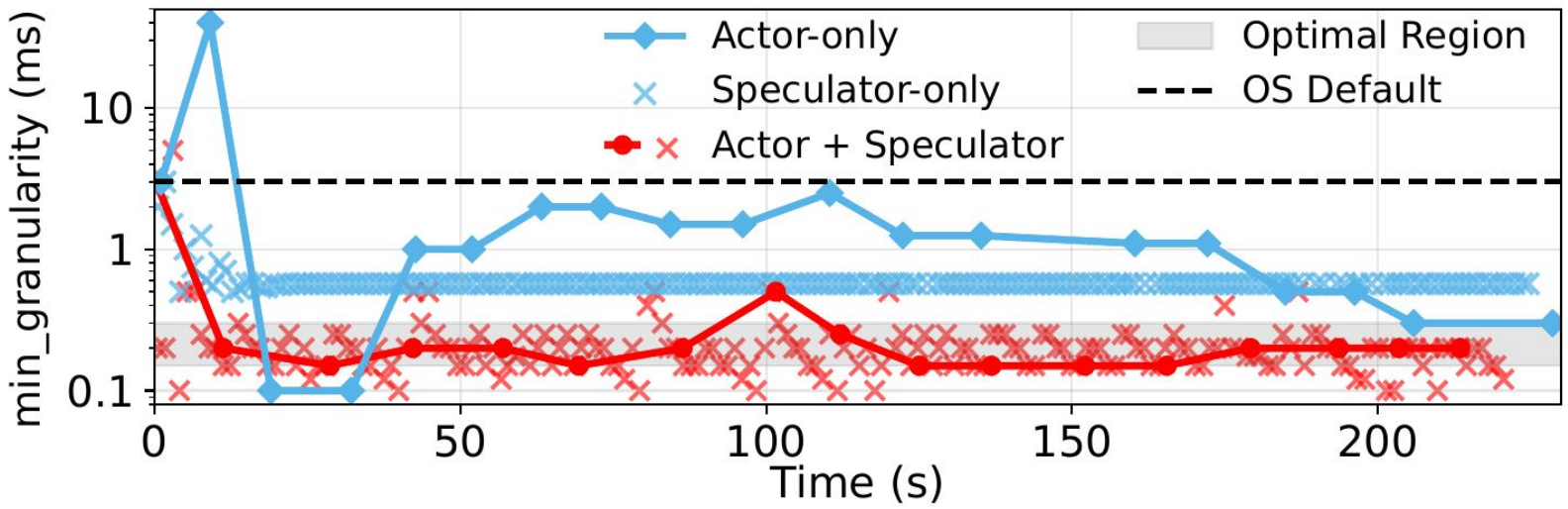
Actor	**LLM tuner deliberates 10–15 s**	**decides**
System	**stuck in a bad setting**	**fixed**

Verifying first would mean waiting for the Actor before every change, so the system stays degraded for 10–15 s each time.

Idea: give up losslessness here

A fast Speculator applies a provisional setting right away. When the Actor finishes, its setting simply overwrites the Speculator's.

Speculator + Actor Find the Optimum in Seconds



1 Escapes bad settings fast

The joint system (red) drops into the optimal band almost at once; Actor-only (blue line) wanders for minutes.

2 Speed alone gets stuck

Speculator-only (blue ×) settles quickly, but above the optimal band.

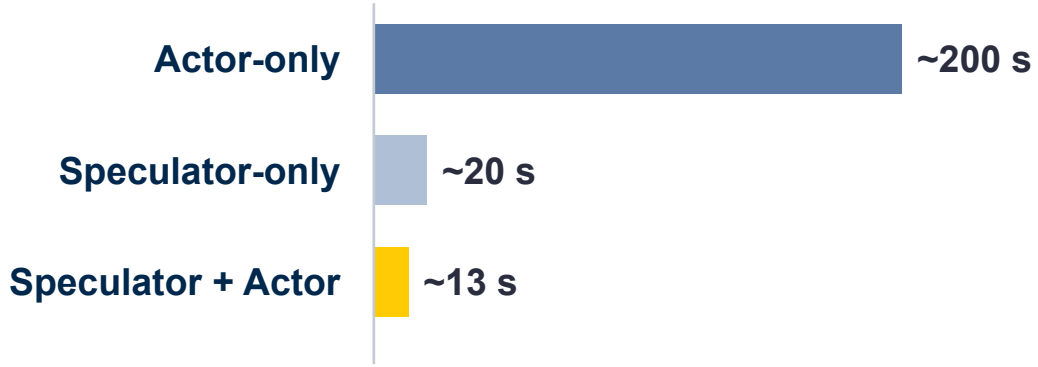
3 Each role covers the other

The Speculator reacts in seconds; the Actor's deeper reasoning steers it to the optimum.

Speculation Pays More per Second, Less in Total

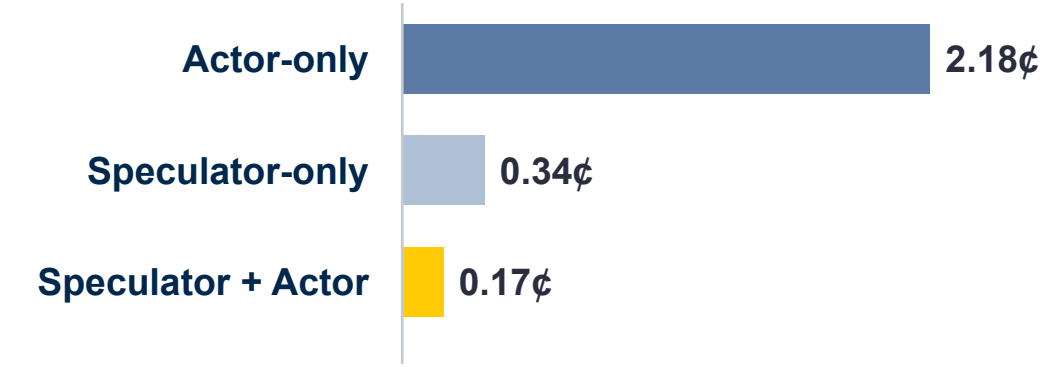
Time to converge

seconds, lower is better



Cost until converged

cents, lower is better



1 Pays more per second

Frequent Speculator calls: at any moment, the joint system has spent more than Actor-only.

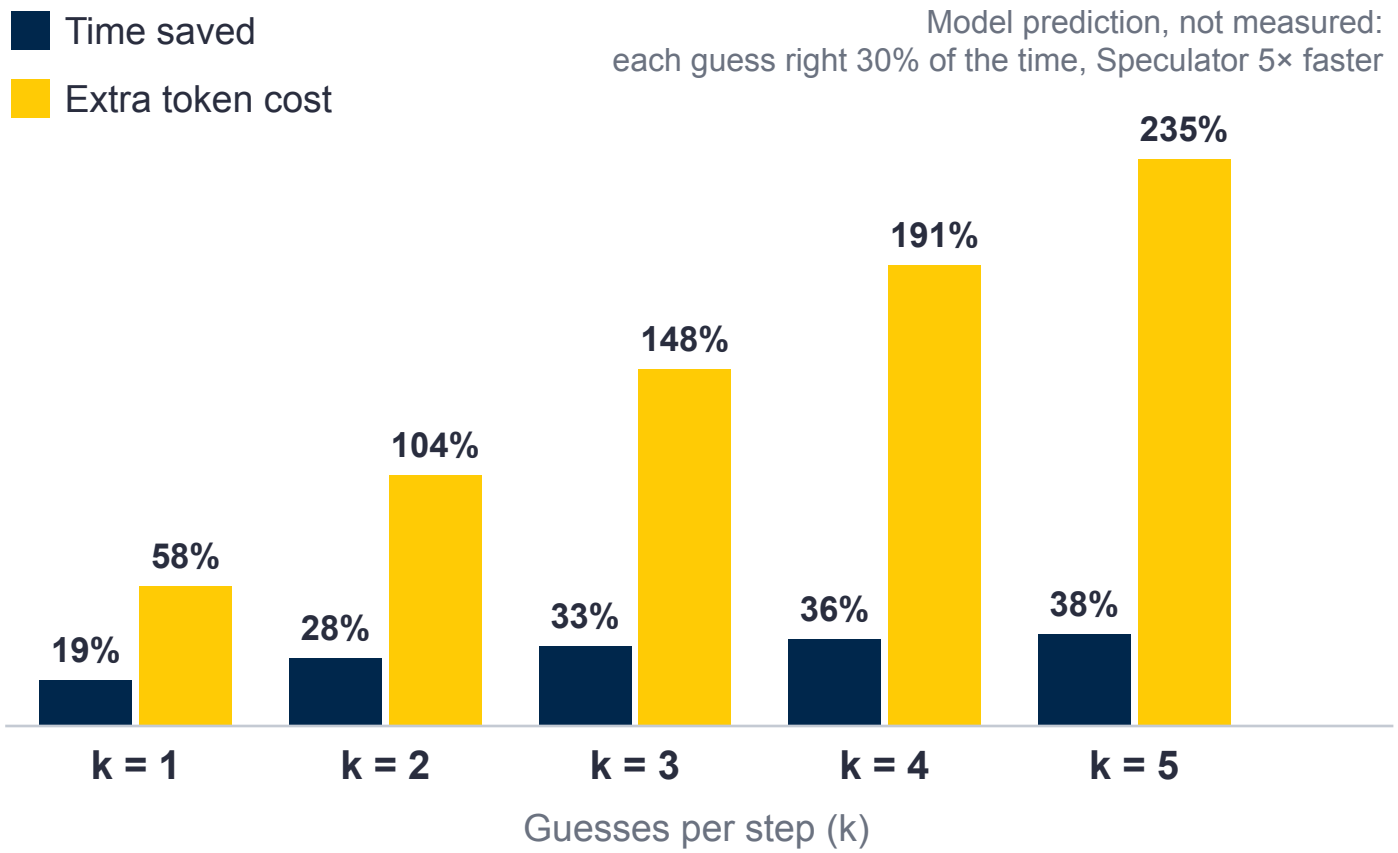
2 Pays less in total

It converges so much sooner that reaching the optimum costs less overall.

3 Only if tuning stops

Left running after convergence, Speculator calls keep adding cost.

Each Extra Guess Saves Less Time and Costs More



1. **Each guess adds less**
   the 2nd guess helps most; by the 5th, time saved barely moves

2. **Every guess costs a call**
   token cost keeps climbing with k

3. **Never more than 50%**
   a step that hits was launched early, so it has no window to speculate on the next one

Takeaways

1. **Agents are slow because every call blocks the next**
   LLM calls, tools and humans run one after another; one run can take an hour
2. **Guess, launch early, verify**
   a cheap Speculator predicts the next call; keep it only if the Actor's real result matches
3. **Next calls are often guessable**
   up to 55% in chess, 38.5% in e-commerce, 46% in multi-hop QA; ~20% less time in chess
4. **If actions can be overwritten, skip verification**
   OS tuning converges in ~13 s instead of ~200 s, and costs less
5. **Extra guesses are not free**
   time saved levels off below 50% while token cost keeps growing

Discussion: Assumptions and Open Questions

Assumptions to question

Lossless speedup timed only once

among lossless settings, only chess reports wall-clock savings; e-commerce and QA report accuracy

Idealized latency model

theory assumes independent, exponential latencies and independent hits

Reversible actions only

orders, emails and deletes can't be speculated safely

Open questions

Semantic matching?

exact match counts equivalent queries as misses; could a looser check stay lossless?

Depth in practice?

multi-step speculation is analyzed only in theory

Shared providers?

k parallel calls assume modest traffic; how do rate limits change the trade-off?

Code: github.com/naimengye/speculative-action

Thank You - Questions?